El **informe técnico** que compartiste (24 de marzo de 2026, F0–F2-S2) refleja un **proyecto muy bien estructurado y profesional** para una app de finanzas familiares como KakeiboPro. La adopción de **Clean Architecture**, **Drift** (offline-first), **Riverpod 2**, **Supabase** y **GoRouter** es una elección sólida y moderna en 2026. El código muestra disciplina: cero issues en `flutter analyze`, atomicidad en transacciones, guards para web, mappers bidireccionales y un enfoque claro en consistencia de datos (spentAmount ajustado atómicamente).

A continuación, un **análisis detallado** con **observaciones positivas**, **áreas de mejora** y **propuestas concretas** (priorizadas por impacto y urgencia). Todo basado en prácticas recomendadas de 2026 para Drift + Supabase + Riverpod en apps multi-tenant familiares.

## 1. Fortalezas destacadas (lo que está muy bien hecho)

- **Clean Architecture bien aplicada**: Separación estricta (Domain sin dependencias externas, Data solo implementa interfaces, Presentation solo consume). Uso de `AppResult<T> + Failure` sealed class + pattern matching → excelente manejo de errores sin excepciones crudas.
- **Atomicidad en Drift**: `saveTransactionAtomic()` con `transaction(() async { ... }) + adjustSpent` es correcto y evita inconsistencias entre `transactions` y `envelopes.spentAmount`. Buen workaround para la ausencia de `Variable.withDateTime`.
- **Gestión de estado con Riverpod**: Uso de `StateNotifierProvider.family + TxKey` público es funcional. Providers reactivos con streams de Drift ( `watchEnvelopes`, `watchByEnvelope` ) → UI actualizada en tiempo real.
- **Offline-first + Sync**: Cola de sincronización ( `SyncQueueTable + WorkManager` ) + guards `kIsWeb` → sólido para Costa Rica (internet inestable).
- **Seguridad inicial**: `sqlcipher_flutter_libs`, `FlutterSecureStorage`, biometría con guards → buena base.
- **Compatibilidad web**: DriftWebOptions + worker + guards → raro verlo tan temprano y bien resuelto.

- **Documentación del informe**: Muy profesional (tabla de sprints, problemas resueltos, deuda técnica, hoja de ruta). Esto facilita mantenimiento.

## 2. Observaciones y Mejoras Técnicas (priorizadas)

### Alta prioridad (hazlo antes de F2-S3)

1. **Migrar Riverpod 2 → Riverpod 3 (o al menos preparar)**
   En 2026, `StateNotifierProvider` y `StateNotifier` son **legacy** (se mantienen por compatibilidad pero se desaconsejan).
   - Cambia a `NotifierProvider` / `AsyncNotifierProvider` (más simples, mejor soporte async).
   - Ejemplo rápido para `AddTransactionNotifier`:

     ```dart
     final addTransactionProvider =
     NotifierProvider.family<AddTransactionNotifier,
     AddTransactionState, TxKey>(
       AddTransactionNotifier.new,
     );

     class AddTransactionNotifier extends
     Notifier<AddTransactionState> {
       @override
       AddTransactionState build(TxKey key) { ... }
     }
     ```

   - Beneficio: menos boilerplate, mejor auto-dispose, menos errores de inicialización síncrona.
   - Acción: Ejecuta `flutter pub upgrade riverpod_annotation` y migra gradualmente (empieza por los notifiers de formularios).

2. **Clave de encriptación SQLCipher (ALTA)**

   Usar timestamp como semilla es inseguro (predecible).

   - Reemplaza por `Random.secure()` + codificación hex (raw key recomendado por Zetetic/SQLCipher):

```dart
import 'dart:math';
import 'dart:convert';

String generateSecureKey() {
  final random = Random.secure();
  final bytes = List<int>.generate(32, (_) =>
random.nextInt(256));
  return 'x\'${bytes.map((b) => b.toRadixString(16).padLeft(2,
'0')).join()}\'';
}
```

   - Almacena la clave generada en `FlutterSecureStorage` la primera vez.
   - Hazlo en `AppDatabase` constructor antes de abrir.

3. **Row Level Security (RLS) en Supabase (ALTA)**
   Pendiente crítico para multi-familia. Sin RLS, cualquier usuario autenticado podría leer/escribir datos de otras familias.
   - Recomendación 2026: Usa **tenant isolation** con `family_id` + JWT custom claim o join a tabla `user_profiles`.
   - Ejemplo policy básica (en Supabase SQL Editor):

     ```
     SQL
     ALTER TABLE envelopes ENABLE ROW LEVEL SECURITY;

     CREATE POLICY "Users can access their family data"
     ON envelopes FOR ALL
     TO authenticated
     USING (family_id = (SELECT family_id FROM user_profiles WHERE
     user_id = auth.uid()))
     WITH CHECK (family_id = (SELECT family_id FROM user_profiles
     WHERE user_id = auth.uid()));
     ```

   - Aplica lo mismo a todas las tablas ( `transactions` , `family_members` , etc.).
   - Prueba con dos cuentas diferentes.
4. **Sincronización offline → Considerar PowerSync o librería dedicada**
   Tu cola manual + WorkManager funciona, pero es propensa a conflictos en apps familiares multi-dispositivo.
   - En 2026, muchas apps Drift + Supabase migran a **PowerSync** (soporte oficial para Drift, realtime, transactions). Reduce código de sync drásticamente.
   - Alternativa ligera: librería `offline_first_sync_drift` o el ejemplo de Musta-Pollo.
   - Acción para F3: Evalúa migrar el SyncRepository si ves conflictos frecuentes.

**Media prioridad (F3–F4)**

- **Tests**: Agrega unit tests para use cases y repositories (mock Drift con `drift_test` ). Widget tests para flujos clave (crear sobre → agregar transacción → ver actualización).

- **Paginación y soft-delete reversión**: Para sobres con muchas transacciones, agrega `.limit(50).offset()` + reversión de `spentAmount` al eliminar.
- **Riverpod best practices**: Usa más `autoDispose` en families para evitar memory leaks. Evita mutaciones síncronas (ya lo manejas con `await Future.value()`, pero en Riverpod 3 es menos necesario).
- **Intl y locales**: `initializeDateFormatting('es')` en main() está bien, pero considera `Intl.defaultLocale = 'es';` y soporte para CRC/USD dinámico.
- **Sentry**: Ya lo tienes en dependencias → configúralo pronto para capturar errores en pruebas reales.

**Baja prioridad / Mejoras de calidad**

- **GoRouter**: Orden de rutas está correcto (estáticas antes de dinámicas). Considera typed routes (go_router 14+).
- **Emoji picker y UI**: Muy bien por ahora. En F3 agrega `fl_chart` para dona de categorías Kakeibo.
- **Web compatibility**: Excelente progreso. Prueba más flujos en Chrome (sync worker).
- **Dependencias**: Revisa `sqlcipher_flutter_libs` (en 2026 tiende a ser obsoleto con sqlite3 v3+; migra si ves warnings).

## 3. Recomendaciones generales para la forma de programar

- **Mantén el ritmo actual**: Clean Architecture + Drift + Riverpod es una combinación potente para apps financieras (type-safe, reactive, testable).
- **Documenta más en código**: Agrega comentarios en métodos atómicos y policies RLS (copia del informe).
- **Próximo sprint (F2-S3)**: Implementa eliminación de transacciones con reversión atómica + commit + prueba en dispositivo real + Chrome.
- **Validación familiar**: Antes de F3 (Reflejos Kakeibo), haz una sesión con tu familia probando el flujo actual. Eso te dará feedback real sobre UX (impacto en spentAmount, preview en formulario).

- **Próximo informe**: Incluye métricas (tiempo de sync, tamaño DB, cobertura de tests) y un diagrama simple de arquitectura (puedes generar con Mermaid).

## Conclusión

El proyecto está en un **excelente punto** para F2. La base es robusta, el código limpio y las decisiones técnicas (atomicidad, guards web, AppResult) son maduras. Los riesgos principales son **seguridad (RLS + clave DB)** y **evolución de Riverpod**. Corrige esas dos ALTA prioridad en los próximos 7–10 días y tendrás una app lista para uso diario familiar.

Si quieres:

- El código exacto para migrar la clave SQLCipher o un ejemplo de RLS policy completo.
- Un prompt listo para Claude/Cursor con las mejoras sugeridas para F2-S3.
- Revisión detallada de un archivo específico (ej. `add_transaction_notifier.dart`).

¡Pégalo aquí y lo analizamos! Estás avanzando muy bien, Carlos. 💪 En unas semanas tendrás el núcleo de KakeiboPro funcionando sólidamente.